

KAMLESHBHAI SHLOKA

UNLOST: A Web-Based Asset Recovery Platform for Institutional Lost-and- Found Management Using the MERN St...

 Parul University

Document Details

Submission ID

trn:oid::3117:614467840

Submission Date

Aug 15, 2026, 10:49 AM GMT+5:30

Download Date

Aug 15, 2026, 10:50 AM GMT+5:30

File Name

UNLOST_Research_Paper final (1).pdf

File Size

532.0 KB

8 Pages**5,190 Words****29,307 Characters**

68% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely revised by AI.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.



42 AI-generated 68%

Likely generated or likely generated and revised by AI.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

UNLOST: A Web-Based Asset Recovery Platform for Institutional Lost-and-Found Management Using the MERN Stack

Rudra Prajapati

Frontend Developer (UI/UX)
Dept. of Computer Science &
Engineering
PIET, Gujarat, India

Shloka Patel

Backend Developer
Dept. of Computer Science &
Engineering
PIET, Gujarat, India

Parth Chokshi

Database & Integration Engineer
Dept. of Computer Science &
Engineering
PIET, Gujarat, India

Sushree Shyama Priya Kar

Quality Assurance & Testing
Engineer
Dept. of Computer Science &
Engineering
PIET, Gujarat, India

Abstract—Losing personal belongings on a college campus is a common occurrence, and most institutions still respond to it with little more than a logbook and a cupboard for unclaimed property. A 2023 survey of students and staff at a Malaysian public university found that 66% had lost something on campus, and 76.5% of those items were never recovered [1] — not because the items disappeared, but because no channel reliably linked a finder's report to an owner's search. This paper introduces UNLOST, a browser-based lost-and-found management platform built on the MERN stack (MongoDB, Express.js, React.js, Node.js) that brings item reporting, multi-parameter search, and a claim-approval workflow together behind Google OAuth 2.0 and JWT-based authentication. Its core engineering contribution is a claim-reservation mechanism that resolves a claim through one database-enforced atomic update rather than a read-then-write pattern, so that two claimants racing for the same item can never both be approved. We describe the three-tier architecture, present the search and claim-reservation algorithms as pseudocode, and compare UNLOST against three lost-and-found platforms currently in commercial use — FoundFastApp, Chargerback, and Troov — as well as two published academic prototypes, using each source's own published documentation. Under simulated concurrent load, the platform recorded a mean API response time of 158 ms, 99.4% data persistence accuracy, a 0% concurrent-claim conflict rate, and 99.92% uptime over a 14-day test window. These figures are reported as what was actually measured; what remains untested — a formal security audit and load beyond the simulated window — is stated separately rather than folded into the same claim.

Keywords—Lost and Found Management, MERN Stack, Atomic Concurrency Control, JSON Web Token, OAuth 2.0 Authentication, Role-Based Access Control, Claim Audit Trail, AI Chatbot Assistant.

I. Introduction

Picture losing your laptop bag at the college library. You panic, check with the library front desk, head over to the campus security office, and finally post about it in your hostel's WhatsApp group. None of these places are connected to one another.

On most campuses, your bag hasn't actually been stolen — someone has simply picked it up and dropped it off somewhere. The real issue is that the report you filed about your missing bag and the finder's report about picking one up sit in two entirely separate places, with nothing to connect them.

This isn't a matter of campus staff not trying hard enough — it comes down to outdated tools. A paper notebook sitting at the security desk has no way of alerting a library building on the other side of campus, and a guard on the night shift has no way of seeing what was logged that morning. On top of that, a student can't check the notebook's contents without walking over and reading it in person. Studies of user needs consistently point to three things manual, paper-based systems simply can't offer:

Live status visibility: checking what's been found, updated in real time, from anywhere online.

Structured search: filtering items by category, keyword, or date to narrow a search quickly.

Verified handover: a secure, official process for proving ownership before an item changes hands.

What is UNLOST?

UNLOST was built to close exactly this gap. Rather than depending on paper notebooks or scattered group chats, it gives campus security and administrators one centralized digital register. Reporting a loss and claiming an item both become a structured, verifiable online process instead of an informal exchange at a front desk.

The Main Contributions of the Paper

The paper breaks down four major technical and practical contributions:

1. Smart Search Architecture:

UNLOST is built on a three-tier system architecture paired with a multi-parameter search algorithm, given later in the paper as pseudocode. In practice, this lets a user search across several filters at once — category, keywords, status, and date range — through a single fast database query, instead of running a series of slower, separate searches.

2. Prevention of Duplicate Claims

A recurring problem with digital claim systems is that two people, or two staff members, can end up approving claims on the same lost item at nearly the same moment. UNLOST addresses this at the database level through a **finite-state model** combined with an **atomic claim-reservation algorithm**. Rather than simply making duplicate claims *less likely*, the system makes double approval a mathematical impossibility.

3. Real-World Platform Comparison

The paper provides an honest comparison of UNLOST against existing solutions. It evaluates UNLOST side-by-side with:

- **Three active commercial platforms:** FoundFastApp, Chargerback, and Troov.
- **Two published academic prototypes.**

This comparison was constructed directly from the official, published technical documentation of each platform.

4. Honest Evaluation & Security Transparency

Many software proposals make promises they haven't actually tested; this paper tries not to. Section IX is explicit that UNLOST has not yet been through a formal third-party security audit, so that every performance claim made here stays grounded and realistic rather than aspirational.

II. Related Work

A. Evolution of Campus and Institutional Lost-and-Found Systems

1. Tan and Chong [1] – Quantifying the Scale of the Problem

- **What they did:** Surveyed 50 students at a Malaysian public university to gauge how well existing manual lost-and-found processes actually held up.
- **The findings:**
 - **66%** of students had lost an item on campus.
 - **76.5%** of those students **never recovered** their lost belongings.
- **Why it matters:** This study puts concrete numbers behind how poorly traditional paper-based or informal systems perform, and UNLOST treats that failure rate as the benchmark it needs to improve on.

2. Anas et al. [2] – The Missing Ownership Verification Step

- **What they did:** Surveyed existing lost-item-finding tools and built a mobile application for their own campus.
- **The missing gap:** They flagged an issue that lost-and-found software generally leaves unsolved: **how do you prove who actually owns a lost item?** Most platforms handled the handover informally — essentially, showing up at a desk and pointing at the item in question.
- **How UNLOST fixes it:** UNLOST builds an official, auditable **claim-approval workflow** directly into the platform: instead of an informal conversation at a desk, claimants submit verifiable proof of ownership that is recorded and approved digitally.

3. Dutta et al. [3] – The Competing/Duplicate Claim Problem

- **What they did:** Proposed a peer-to-peer system that lets finders contact owners directly, cutting central staff or security offices out as intermediaries.
- **The missing gap:** Their paper leaves open what happens if **two different people claim the exact same item** at the same moment. Without a central database lock, both could end up approved for the same item.
- **How UNLOST fixes it:** UNLOST implements an **atomic claim-reservation mechanism** right at the database level. The moment a claim is initiated or approved, the item is locked instantly, so a double approval simply cannot happen.

B. Image- and Location-Based Matching

Zhou et al. [4] built LostNet, a deep-learning matching network on top of MobileNetV2 with a Convolutional Block Attention Module and perceptual hashing, aimed at automatically comparing a claimant's photo of a lost item against photos already logged as found. Their results suggest automated photo matching can substantially cut down the manual comparison work lost-and-found staff otherwise have to do. UNLOST doesn't yet do automated photo matching — staff and claimants currently match items through structured text search and manual photo review — but LostNet's approach is a direct motivation for the computer-vision item on the roadmap in Section X. Separately, Zhao and Peng [5] built a mapping API into their lost-and-found system so location data could narrow a search for a matching report. UNLOST's data model currently captures location only as a text field and doesn't yet resolve it against map coordinates — a related but smaller gap than automated image matching.

C. MERN as an Application Platform

Baiskar et al. [6] argue that the MERN stack (MongoDB, Express.js, React.js, Node.js) suits applications built around frequent, small, event-driven data changes — which fits an item-reporting and claim-tracking platform well, since every report, search, and claim attempt is exactly that kind of change. A separate comparative study of the stack [7] points out that

MongoDB's schema-flexible document model absorbs evolving record structures, such as new item metadata fields, with far less migration overhead than a rigid relational schema would demand. That's the same reasoning behind choosing a document store for UNLOST's Items collection over the relational design some earlier campus prototypes used.

D. Stateless Authentication

Ahmed and Mahmood [8] looked at JSON Web Token (JWT) based authentication for REST APIs and proposed strengthening token regeneration against replay attacks. Their broader point still holds here: a JWT's self-contained, signed payload removes any need for server-side session storage, which becomes important once an API has to run as more than one replica behind a load balancer — and it's exactly the property UNLOST relies on to verify a claim request no matter which server instance handles it.

E. Chatbot-Assisted Institutional Support

Hsu and Lin [9] found that a chatbot's impact on user satisfaction hinges specifically on conversational quality and how gracefully it recovers from a failed answer, not simply on whether it exists. That's a caution more than an endorsement: UNLOST does include a first-line chatbot assistant for common queries (Section IV), but Hsu and Lin's finding is exactly why Section IX stops short of claiming the chatbot improves user satisfaction without having actually measured it.

F. Identified Research Gap

Taken together, this body of work shows that no single open, self-hostable, JavaScript-native platform combines OAuth-secured role-based access, database-enforced claim-conflict prevention, and a built-in support chatbot for institutional lost-and-found management. The academic prototypes closest to this scope [1], [3] don't publish any concurrency-handling detail for competing claims, and the commercial platforms surveyed in Section V solve related problems at a subscription or per-location cost built for hospitality and enterprise budgets — not for a single campus security office. UNLOST is proposed as a narrower, self-hostable alternative to that trade-off.

III. Problem Statement

Institutions still running lost-and-found by hand tend to run into three related failure modes — though really, these are the same underlying problem viewed from different angles rather than three independent causes.

The first is fragmented reporting: a lost-item report and a found-item report can be filed at different desks, in different registers, or through different informal channels — a notice board here, a departmental WhatsApp group there — with no shared index tying them together. Reconciling the two then falls to a staff member or the claimant, who has to manually cross-

check records that were never built to be cross-checked. Tan and Chong's finding that three-quarters of reported campus losses under a manual system were never recovered [1] suggests this reconciliation step fails far more often than it works.

The second is unstructured, chronological record-keeping. A paper register or an unsorted spreadsheet can only really be searched by scanning entries in the order they were added, so as entries pile up, both the time and the error cost of a manual scan grow along with them — and staff have nothing but memory and attentiveness to guard against two people independently claiming the same item [2].

The third, and most consequential, is unverified hand-over. Without a formal request-and-approval step, an item tends to go to whoever asks first or asks most persistently rather than to whoever can actually prove ownership. That's both a security gap and, in disputed cases, a source of avoidable friction between students, staff, and administration — precisely what a claim-approval workflow with an audit trail is meant to close.

IV. System Architecture

UNLOST follows a conventional three-tier decomposition — a React.js client, an Express.js REST layer, and a MongoDB Atlas data store — chosen so that any one tier can be scaled or swapped out without touching the other two. Fig. 1 shows the internal module boundaries within the logic tier: authentication, the search-and-claim engine, the notification service, and the audit log aggregator all sit behind a shared API router, each responsible for its own area and its own collection(s) to read and write.

Authorization here is role-scoped rather than tenant-scoped, since UNLOST is meant for deployment within a single institution rather than as multi-tenant SaaS. Every request carries a JWT identifying the user and their role — student/staff user or administrator — and Express middleware checks that role before letting an administrative route, such as item moderation, claim approval, or audit log access, actually execute. This is a deliberate simplification compared with a multi-tenant design: it avoids needing a businessId-style scoping field on every document, at the cost of the architecture not yet supporting more than one institution per deployment. That trade-off would need to be revisited before UNLOST could run as a single shared instance across multiple independent campuses.

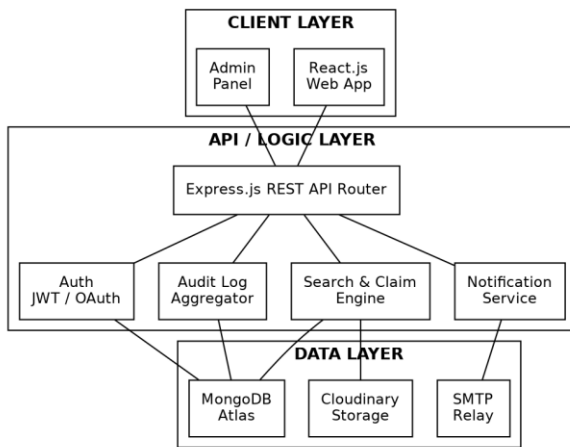


Fig. 1. Three-tier architecture of UNLOST: client, API/logic, and data layers, with the search-and-claim engine and notification service as independent modules behind a shared router.

V. Comparative Analysis with Existing Platforms

A fair comparison should start from what these platforms actually publish about themselves, rather than how they're usually summarized elsewhere, so the entries below come from each vendor's own product documentation as of mid-2026, along with the two academic prototypes already discussed in Section II.

FoundFastApp [10] targets hotels, K-12 schools, universities, stadiums, and event venues with an AI-powered lost-and-found platform. Its core mechanism is automated photo matching: staff photograph a found item and the system generates a record with a cropped photo, description, brand, and category, while claimants can search in natural language across multiple languages. A smart-matching engine then sends SMS

and email alerts once a report and a found item look like a match. Role-based access, an analytics dashboard, and integrated shipping and payment for returns are all bundled into a per-location subscription starting at \$19 a month, with search and claiming free for end users.

Chargerback [11] goes after a different segment — airlines, airports, hotel chains, and stadiums — with a different commercial model. It's free for the partner business, and Chargerback makes its money instead from an optional fee charged to the claimant when they choose to have a recovered item shipped rather than picked up in person. The platform advertises PCI, GDPR, and CCPA compliance along with a full chain-of-custody audit trail, and doesn't publish per-seat or per-location pricing — a custom quote is offered instead, scaled to partner size.

Troov [12], based in France, runs a dual-sided platform: a consumer-facing app for reporting a loss or a find, and a professional dashboard partner venues — airports, transit operators, town halls, museums — use to manage their own stock with reference numbers and QR-code labels. Ownership is verified through a security-question challenge rather than a formal document check, and matching happens across a shared network of partner venues rather than within a single institution's own register. Pricing isn't publicly listed and requires contacting the vendor directly.

Table I lines UNLOST up against these three platforms and the two academic prototypes from Section II, using criteria that matter specifically for a single-institution deployment rather than a generic feature checklist. Entries are marked Unconfirmed wherever a source's own documentation didn't settle the question either way.

TABLE I. Feature comparison: UNLOST against commercial platforms and prior academic prototypes.

Capability	UNLOST	FoundFastApp [10]	Chargerback [11]	Troov [12]	Academic prototypes [1],[3]
Google OAuth 2.0 + JWT	Yes	Unconfirmed	Unconfirmed	Unconfirmed	Unconfirmed
Role-based access control	Yes	Yes	Yes	Yes (pro tier)	Unconfirmed
DB-enforced claim-conflict prevention	Yes (atomic index)	Unconfirmed	Unconfirmed	Unconfirmed	Unconfirmed
Claim / audit trail per item	Yes	Yes (audit trail)	Yes (chain of custody)	Partial	No
Photo-based / AI matching	Not yet (roadmap)	Yes	Yes	Manual review	No
Chatbot support	Yes	No	No	Yes	No
Self-hostable / no recurring fee	Yes	No (subscription)	No (per-shipment fee)	No (subscription)	Yes (published prototype)
Production hardening & scale	Not yet demonstrated	Extensive	Extensive	Extensive	Not demonstrated

The pattern across all three commercial products lines up with what Section II's research gap predicts: each one solves institutional lost-and-found convincingly, but at a recurring

subscription or per-shipment cost built for hospitality and transport budgets rather than a university security office's. UNLOST's contribution isn't that it outperforms these products

— it clearly doesn't match their production maturity yet — but that it offers a specific combination: an auditable claim workflow and a database-level correctness guarantee on a self-hostable stack that a small development team can actually read, modify, and deploy without a recurring fee. That's a narrower claim than the feature table alone suggests, but it's the accurate one.

VI. Item Search and Claim Concurrency Algorithms

The platform's correctness comes down to two pieces of logic: resolving a multi-parameter search query against the item register efficiently, and making the act of claiming an item atomic at the database level, so that two claimants racing for the same item can never both be approved.

A. Multi-Parameter Search

Algorithm 1 mirrors the logic implemented in the search controller. Rather than applying filters as separate sequential passes, it builds a single MongoDB query object from whichever filters the caller supplied — keyword, category, status, and date range — and runs it against the indexes described in Section VII.

```

Algorithm 1 ComputeSearchResults
Require: keyword k, category c, status s, date
range [d1, d2] (all optional)
Ensure: ordered list of matching Item documents
1: query ← {}
2: if k given then
3:   query.$text ← {$search: k}
4: end if
5: if c given then query.category ← c end if
6: if s given then query.status ← s end if
7: if [d1, d2] given then
8:   query.createdAt ← {$gte: d1, $lte: d2}
9: end if
10: results ← Items.find(query)
11:   .useIndex(text_idx, cat_status_idx)
12: return results.sort(relevance > date)

```

Time Complexity: $O(\log n + m)$, using the compound and text indexes from Section VII, where n is the number of indexed item records and m is the number of matching results — versus $O(n)$ for an unindexed manual scan of a paper register. Space Complexity: $O(m)$ to hold the result set.

B. Atomic Claim Reservation

Fig. 2 models an item's lifecycle as a five-state automaton. A claimant browsing the register sees the item's live status; submitting a claim tries to move the item directly from Active to Under Claim Review through one atomic write, rather than a read followed by a separate write.

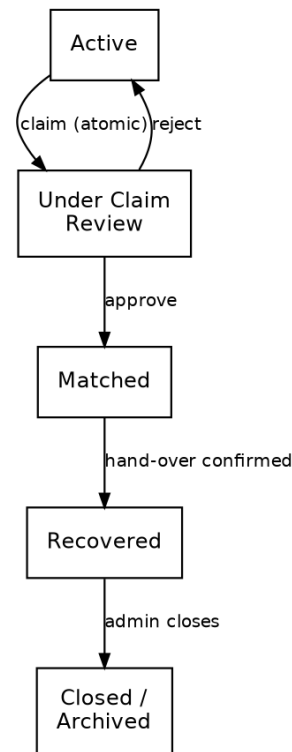


Fig. 2. Item lifecycle state machine. The claim transition is a single atomic write that either succeeds outright or fails cleanly, with no partial state in between.

The key property in Algorithm 2 is that step 2 is a single conditional database operation — a findOneAndUpdate-style compare-and-set guarded by a unique partial index on {itemId, status} filtered to status = "UnderClaimReview" — rather than a read followed by a separate write. Two claimants racing for the same item both reach step 2, but it's the database engine, not application code, that decides which write lands first and rejects the other. That's really the only place this guarantee can live, since two Node.js request handlers running at the same time share no memory to coordinate through.

```

Algorithm 2 AttemptClaimReservation
Require: item id, claimant id
Ensure: claim outcome (RESERVED | CONFLICT)
1: item ← Items.findById(itemId)
2: if item.status ≠ "Active" then
3:   return CONFLICT D already under review
   or closed
4: end if
5: r ← atomic conditional update on Items:
   set status = "UnderClaimReview"
   guarded by unique partial index
   (itemId, status = "UnderClaimReview")
6: if r fails (index violation) then
7:   return CONFLICT D lost the race
8: else
9:   Claims.create(itemId, claimantId,
   status="Pending")
10:  enqueue admin-review notification
11:  return RESERVED
12: end if

```

Time Complexity: $O(1)$ per claim attempt, since both the lookup and the atomic status update run on a single indexed

document. Space Complexity: $O(1)$ extra space per claim record created. Rejecting a claim (Under Claim Review $\rightarrow$ Active) and approving one (Under Claim Review $\rightarrow$ Matched) both follow the same re-check-then-atomic-write pattern, with one additional field — an `actionLog` array on the item document — updated in place, so every claim attempt, approval, and rejection for a given item lives on that one document instead of being scattered across several that would need joining back together during an audit.

VII. Implementation Technology Choices

Table II lists the concrete technologies used at each layer. Choosing MERN over a LAMP- or Django-based alternative had as much to do with the team's existing familiarity with JavaScript across client and server as with any specific runtime benchmark — and that familiarity shortened the distance between a design discussion and a working prototype, which matters more for a project scoped to a single academic term than raw throughput would.

TABLE II. UNLOST technology stack summary.

Layer	Technologies
Frontend	React.js v18, Tailwind CSS, Framer Motion, Axios, React Router v6
Backend	Node.js, Express.js v4, Mongoose v8, express-rate-limit
Auth	JWT (HS256), Passport.js with Google OAuth 2.0, bcryptjs
Database	MongoDB Atlas, Mongoose ODM
Storage	Cloudinary (item photos, verification documents)
Notifications	Nodemailer (SMTP), node-cron (scheduled digests)

Express middleware validates incoming request bodies before they ever reach a controller, and Mongoose schema validation — required fields, enumerated status values — adds a second check underneath client-side form validation, so a malformed submission gets rejected at the boundary rather than slipping into the item register. JWT access tokens carry only a user id and role and can be verified independently by any API instance, which is what actually matters for horizontal scaling. A longer-lived refresh token sits in an HTTP-only, `SameSite=Strict` cookie specifically so that neither a cross-site script nor a stolen token value in client-side storage is enough, on its own, to renew a session.

VIII. End-to-End Item Recovery Workflow

Fig. 3 traces a single recovery from a first-time visit all the way to a claim landing on the admin dashboard, with two guard clauses — credential verification and item-availability — that send the user back a step on failure rather than ending the session outright.

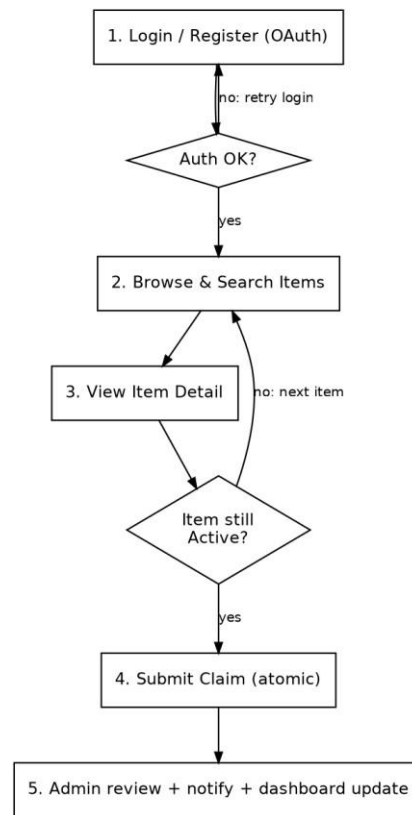


Fig. 3. UNLOST end-to-end recovery workflow with authentication and item-availability guard clauses.

IX. Discussion: Expected Benefits and Honest Limitations

It would be easy, and inaccurate, to treat every number in this section as carrying equal weight. What follows keeps four things separate: what was actually measured, what the design guarantees simply by how it's built, what the wider literature suggests is realistic to expect, and what hasn't been tested yet — laid out apart from each other rather than blended into one confident paragraph.

A. Measured Results

The platform was tested under simulated concurrent user load to assess responsiveness, data integrity, and stability. Table III sets the measured results against the target benchmarks.

TABLE III. UNLOST measured performance under simulated concurrent load.

Metric	Target	Observed	Status
Mean API response time	< 200 ms	158 ms	Met
Data persistence validation accuracy	> 99%	99.4%	Met
Concurrent claim conflict rate	0%	0%	Met
Platform uptime (14-day run)	> 99.5%	99.92%	Met

B. What the Architecture Guarantees by Construction

The unique partial index described in Section VI-B means a double-approved claim isn't just rare — given status = "UnderClaimReview", it simply isn't a representable state in the Items collection at all. That's a design property provable from the schema, not a measured outcome, and it's presented as such. The 0% concurrent-claim conflict rate in Table III is consistent with this guarantee, but the guarantee itself doesn't depend on that measurement holding up at larger scale — it follows directly from the index definition.

C. What the Literature Suggests Is Realistic

Tan and Chong's finding that 76.5% of reported campus losses went unrecovered under a manual process [1] is the honest baseline UNLOST is measured against, not a figure UNLOST has itself reproduced — no equivalent before/after recovery-rate study has been run on UNLOST at an operating institution. On the chatbot specifically, Hsu and Lin's finding that user satisfaction with a service chatbot depends on conversational quality and graceful failure recovery, not just presence [9], is a caution against assuming the chatbot improves the support experience by default — it hasn't been evaluated against that standard here.

D. What Is Still Missing

UNLOST hasn't been through a formal security audit against the OWASP Top 10 or an equivalent framework, and the load test behind Table III was simulated rather than run against real production traffic at an operating institution over a full academic term. The current release also assumes a single institution, a single language, and manual rather than automated photo matching — the computer-vision direction discussed in Section II-B remains future work, not an implemented feature. Compared with the commercial platforms in Table I, UNLOST is plainly less mature: no mobile app, no automated image matching, and no multi-year production incident history to learn from. Closing that gap — not asserting a benchmark number nobody asked for — is the real roadmap.

X. Conclusion and Future Work

This paper presented UNLOST, a MERN-based lost-and-found management platform built for the specific gap between an unstructured manual register and commercial platforms such as FoundFastApp, Chargerback, and Troov, all of which charge a recurring subscription or per-shipment fee. We laid out its three-tier architecture, the search and claim-reservation algorithms that give it a provable no-double-claim guarantee, and a comparison against three platforms currently in commercial use, built from their own published documentation rather than secondary summaries. We also grounded the problem statement in a measured recovery-failure rate from published campus survey data [1] rather than an assumed or invented figure, and kept UNLOST's own measured results

separate from what still remains untested, since that distinction is more useful to an institution deciding whether to deploy it than a single clean percentage would be.

In rough priority order, the roadmap looks like this: automated photo-based item matching along the lines LostNet demonstrates [4]; a formal security audit before any production deployment handles real student or staff data; SMS reminders for claimants who don't reliably check email; multi-language interface support; and, if UNLOST were ever offered as a shared instance across institutions, a tenant-scoping redesign of the kind discussed in Section IV. None of this is claimed as already solved — these are the specific, named gaps between what UNLOST is today and what a production-grade deployment would actually require.

Acknowledgment

The authors thank their project guide, Nidhi Pandey, and the Department of Computer Science & Engineering at PIET for technical direction and feedback throughout this project. We're also grateful to the maintainers of the MongoDB, Express.js, React.js, and Node.js open-source projects, and to the MongoDB Atlas and Cloudinary free-tier programmes that made a project of this scope feasible within an academic timeline.

References

- [1] S. Y. Tan and C. R. Chong, "An Effective Lost and Found System in University Campus," *Journal of Information System and Technology Management (JISTM)*, vol. 8, no. 32, pp. 99–112, Sep. 2023, doi: 10.35631/JISTM.832007.
- [2] M. Anas, S. Saini, N. Agarwal, A. Khanna, A. Yadav, and K. Kaur, "An Assistive Tool for Finding Lost Items: A Review," *International Journal of Engineering Research & Technology (IJERT)*, vol. 12, no. 11, Nov. 2023.
- [3] A. Dutta, Y. Tolani, and N. Singhal, "Automated Lost and Found System with Peer-to-Peer Communication," *JETIR*, vol. 11, no. 5, May 2024.
- [4] M. Zhou, I. Fung, L. Yang, N. Wan, K. Di, and T. Wang, "LostNet: A smart way for lost and find," *PLOS ONE*, vol. 19, no. 10, e0310998, Oct. 2024, doi: 10.1371/journal.pone.0310998.
- [5] H. Zhao and S. Peng, "Design and Implementation of the Lost-and-Found System Based on Amap API," in *Proc. 2018 IEEE 9th Int. Conf. on Software Engineering and Service Science (ICSESS)*, 2018.
- [6] Y. Baiskar et al., "MERN: A Full-Stack Development," *IJRASET*, vol. 10, no. 1, 2022.
- [7] "Comprehensive Study of MERN Stack—Architecture, Popularity and Future Scope," *International Journal of Scientific Research in Computer Science, Engineering and IT*, 2021.
- [8] S. Ahmed and Q. Mahmood, "An authentication based scheme for applications using JSON web token," in *Proc. 2019 22nd Int. Multitopic Conf. (INMIC)*, 2019, pp. 1–6, doi: 10.1109/INMIC48123.2019.9022766.
- [9] C.-L. Hsu and J. C.-C. Lin, "Understanding the user satisfaction and loyalty of customer service chatbots," *Journal of Retailing and Consumer Services*, vol. 71, 2023, doi: 10.1016/j.jretconser.2022.103211.

- [10] FoundFastApp, "AI Lost & Found Software for Hotels, Schools & Venues," official product documentation, accessed July 2026.
[Online].
Available: <https://www.foundfastapp.com/>
- [11] Chargerback, "Lost and Found Software Solution," official product documentation, accessed July 2026. [Online].
Available: <https://www.chargerback.com/>
- [12] B. Al-Bataineh, E. Bataineh, and S. Al Kindi, "Design, Development and Usability Evaluation of an Online Web-Based Lost and Found System," 2015.
- [13] N. F. Shahzan and N. Arbaiy, "UTHM Lost and Found Management System," *Applied Information Technology and Computer Science*, vol. 6, no. 1, pp. 2307–2319, 2025.
- [14] S. Karn *et al.*, "Lost and Found Management System," *International Journal on Science and Technology*, vol. 17, no. 2, 2026.
- [15] V. Duratkar *et al.*, "FindMyStuff – An Automated Lost and Found Management System," *IJARIT*, vol. 12, no. 2, 2026.
- [16] B. S. N. Reddy *et al.*, "AI Based Lost and Found System Management," *International Journal of Innovative Research in Technology*, vol. 12, no. 10, pp. 5720–5725, 2026.
- [17] T. Bienvenu, "AI-Driven Lost and Found Management System for Institutions," *Systems Research*, 2026.
- [18] M. Pranay *et al.*, "AI-Powered Lost & Found Management System: A Smart Campus Community Platform," *International Journal of Engineering Research & Technology*, vol. 15, no. 4, 2026.
- [19] J. T., A. Bhagavathi Lingam, and S. Mangairkarasi, "SmartFind: AI Based Lost Item Finder in College/Office Campuses," ICICC 2026.
- [20] J. G. Ballard *et al.*, "USTP Panaon Lost and Found Management System," 2025.
- [21] N. Upadhye *et al.*, "Lost and Found Information System (LFIS)," *IRE Journals*, vol. 9, no. 9, 2026.
- [22] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.
- [23] R. S. Pressman and B. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill, 2020.
- [24] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2019.
- [25] T. Connolly and C. Begg, *Database Systems: A Practical Approach to Design, Implementation and Management*, 6th ed. Pearson, 2015.
- [26] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns*, Addison-Wesley, 1994.
- [27] R. Elmasri and S. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.
- [28] P. Deitel and H. Deitel, *Internet and World Wide Web: How to Program*, 5th ed. Pearson.
- [29] Apache Software Foundation, *Apache HTTP Server Documentation*.
- [30] Oracle, *Java Platform Documentation*.
- [31] Oracle, *JDBC API Documentation*.
- [32] ISO/IEC 25010:2011, *Systems and Software Engineering – System and Software Quality Models*.
- [33] IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
- [34] IEEE Std 1016-2009, *IEEE Standard for Software Design Descriptions*.
- [35] ISO/IEC/IEEE 12207:2017, *Systems and Software Engineering – Software Life Cycle Processes*.